# Module 7: Developing Event-Driven Serverless Solutions

AWS Academy Cloud Developing

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Section 1: Introduction

## Module 7: Developing Event-Driven Serverless Solutions

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Module objectives

At the end of this module, you should be able to:

- Explain serverless computing
- Describe how AWS Lambda works
- Recognize Lambda invocation models
- Identify how to use AWS Identity and Access Management (IAM) to grant Lambda permissions
- Indicate the steps to author and configure a Lambda function
- Explain how to deploy Lambda functions
- Identify why AWS X-Ray is a critical developer tool
- Create Lambda functions

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Module overview

## Sections

1. Introduction
2. Introducing serverless computing
3. Introducing Lambda
4. Invoking Lambda functions
5. Setting permissions for Lambda
6. Authoring and configuring Lambda functions
7. Deploying Lambda functions
8. Monitoring and debugging tools for application developers

## Demonstration

Using X-Ray with Lambda

## Lab

Creating Lambda Functions Using the AWS SDK for Python

Knowledge check

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

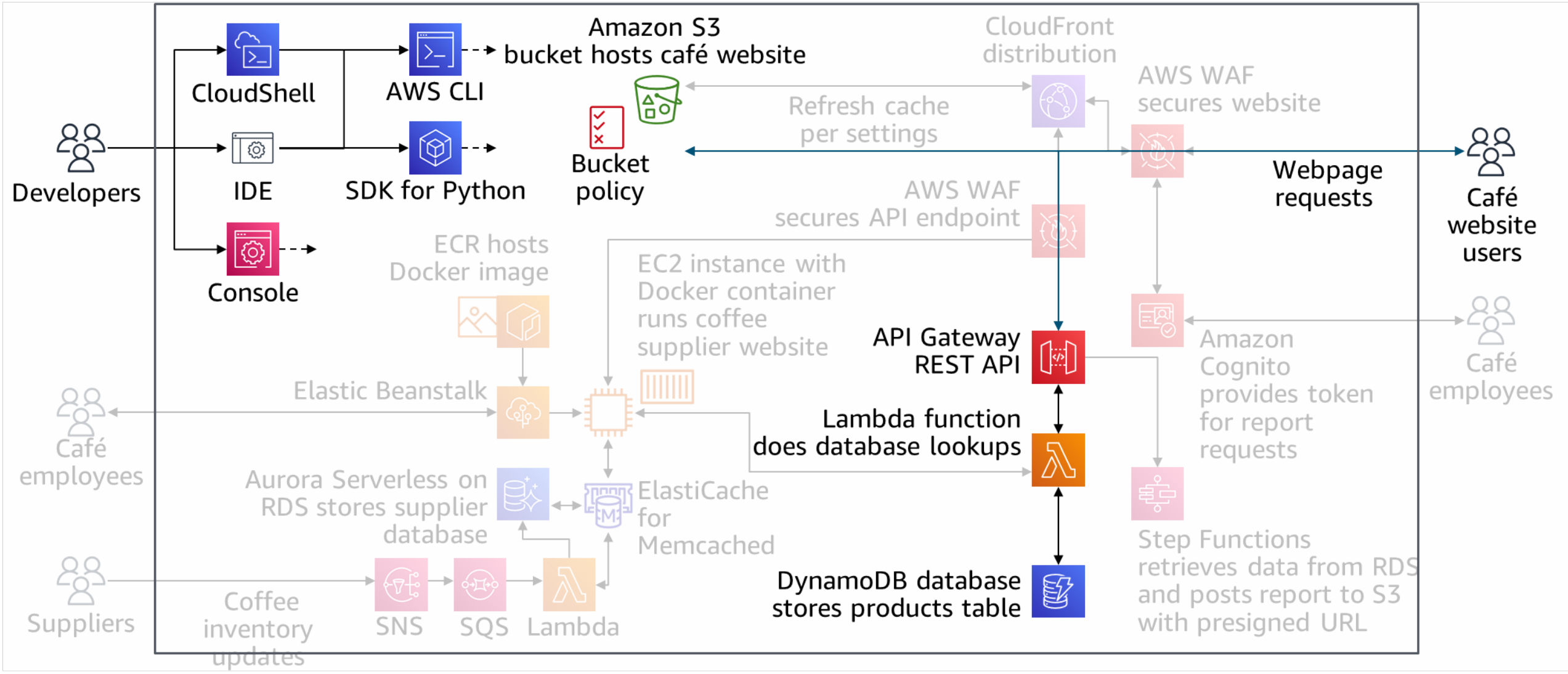
# Café business requirement

Sofía finished using the mock endpoints to test the API, and now she wants to launch the dynamic version of the café website. She needs to update the API integration to connect to the backend database.

```
cursor.close()

sql = "INSERT INTO hive (dev_id, humidity, temperature) VALUES (%s, %s, %s)"

def on_connect(client, userdata, flags, rc):
    print("Connected with result code " + str(rc))

    client.subscribe("+/devices/+/up")

def on_message(client, userdata, msg):
    print(msg.topic + " 666 " + str(msg.payload)+"\n")
    data = json.loads(msg.payload.decode('utf8').replace("'", '"'))
    val = (data["dev_id"], data["payload_fields"]["humidity"], data["payload_fields"]["temperature"])
    connection = mysql.connector.connect(
        host="bees-mariaDB",
        user="bees",
        passwd=mysql_password,
        database="bees"
    )
    cursor = connection.cursor()
    cursor.execute(sql, val)
    connection.commit()
    print(mycursor.rowcount, "record inserted.")
    cursor.close()
    connection.close()

client = mqtt.Client()
```

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Serverless as part of developing a cloud application




©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Section 2: Introducing serverless computing

## Module 7: Developing Event-Driven Serverless Solutions

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Serverless computing

Level of abstraction

Focus on business logic

## On premises

- Heavy capital expense
- Guessed capacity
- Deployed in months
- Years of maintenance
- Low innovation factor

## Virtual servers in the cloud

- Hardware independence
- Elastic resources and scale
- Faster provisioning
- Reduced maintenance

## Containers

- Platform independence
- Higher resource utilization
- Easier and faster deployments
- Isolation and sandboxing

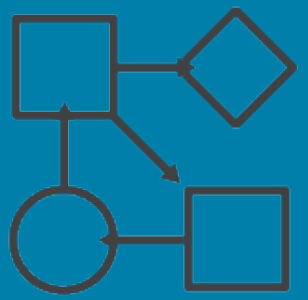
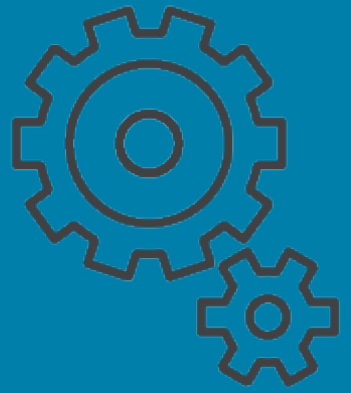
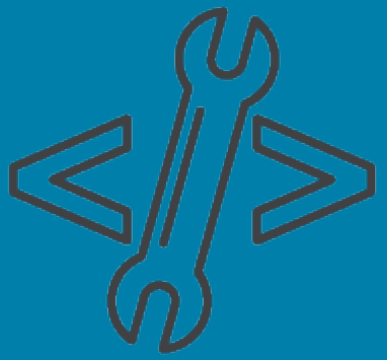
## Serverless

- No OS instances to manage
- Flexible scaling
- Pay per usage
- Built-in fault tolerance
- Zero maintenance

aws

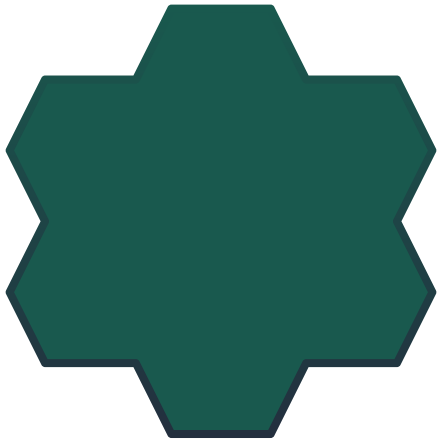
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Building modern serverless applications

| Small pieces, loosely joined | Purpose-built data stores | Specialized services for integration | Automation by using code |
| --- | --- | --- | --- |
|  | |  |  |
| Serverless compute and event-driven patterns | Serverless, purpose-built data stores | Managed services for integration | Deployment frameworks |

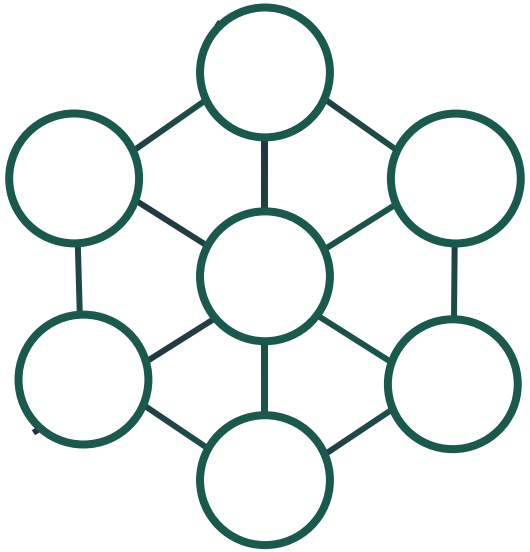
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Moving away from a monolithic application



Monolith

Does everything



Microservice

Does one thing

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Section 3: Introducing Lambda

## Module 7: Developing Event-Driven Serverless Solutions

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# AWS Lambda

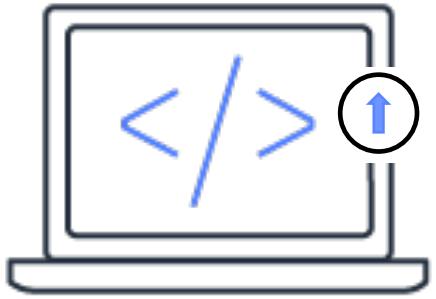
AWS Lambda

Lambda is a serverless compute service that you can use to run code without provisioning or managing servers.

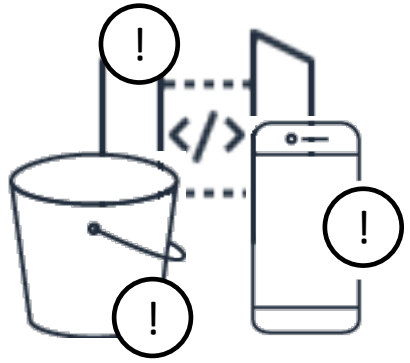
- Invokes your code in response to events
- Scales automatically
- Provides built-in code monitoring and logging with Amazon CloudWatch

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# How Lambda works



Upload your code to Lambda or write code in the Lambda editor



Set up your code to run when events occur in other AWS services, at HTTP endpoints, or as part of in-app activity

Lambda runs your code only when it is activated by an event and uses only the compute resources that are needed

Pay only for the compute time that you use

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

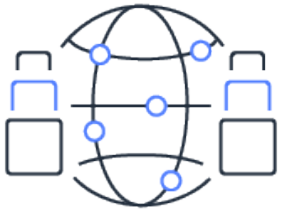
# Lambda features

Offers the ability to bring your own code

Integrates with and extends other AWS services

Offers flexible resource and concurrency models

Offers a flexible permissions model



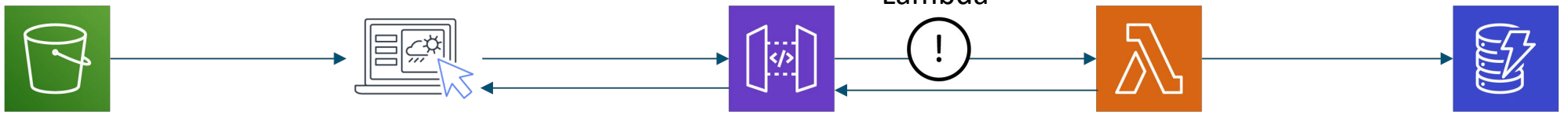
Provides built-in availability and fault tolerance

Reduces the need to pay for idle resources

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Example use cases for Lambda

Web app




Amazon S3

The front-end code for the weather app is hosted on Amazon S3

Website

A user chooses a link for local weather information

API Gateway

The app makes a REST API call to the endpoint

Lambda

Lambda runs code to get the weather data from DynamoDB and returns the data

DynamoDB

DynamoDB contains the weather data that the app uses

Real-time stream processing

Kinesis

A social media stream is loaded into Kinesis in real time

Lambda

Lambda runs code that generates hashtag trend data

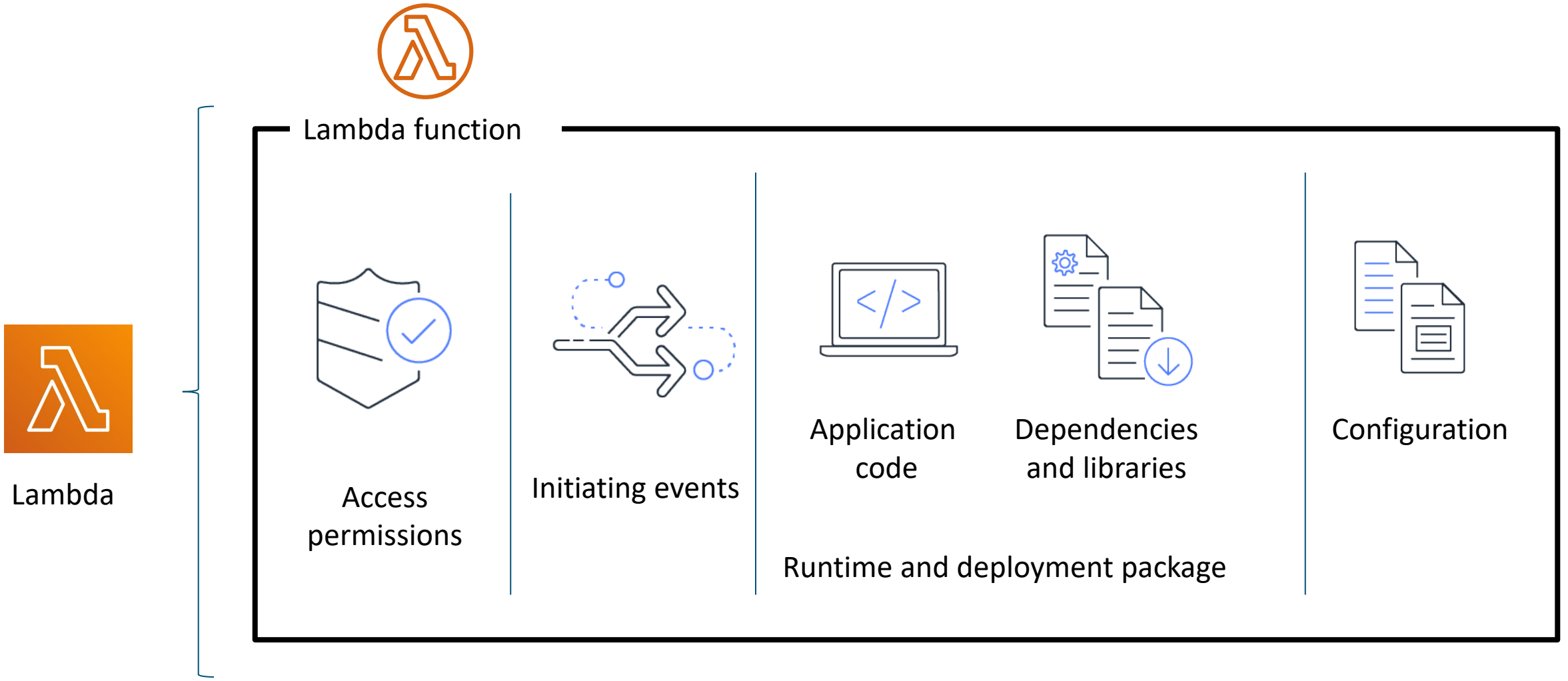
DynamoDB

Hashtag trend data is stored in DynamoDB

Social media trend data is immediately available for business users to query

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

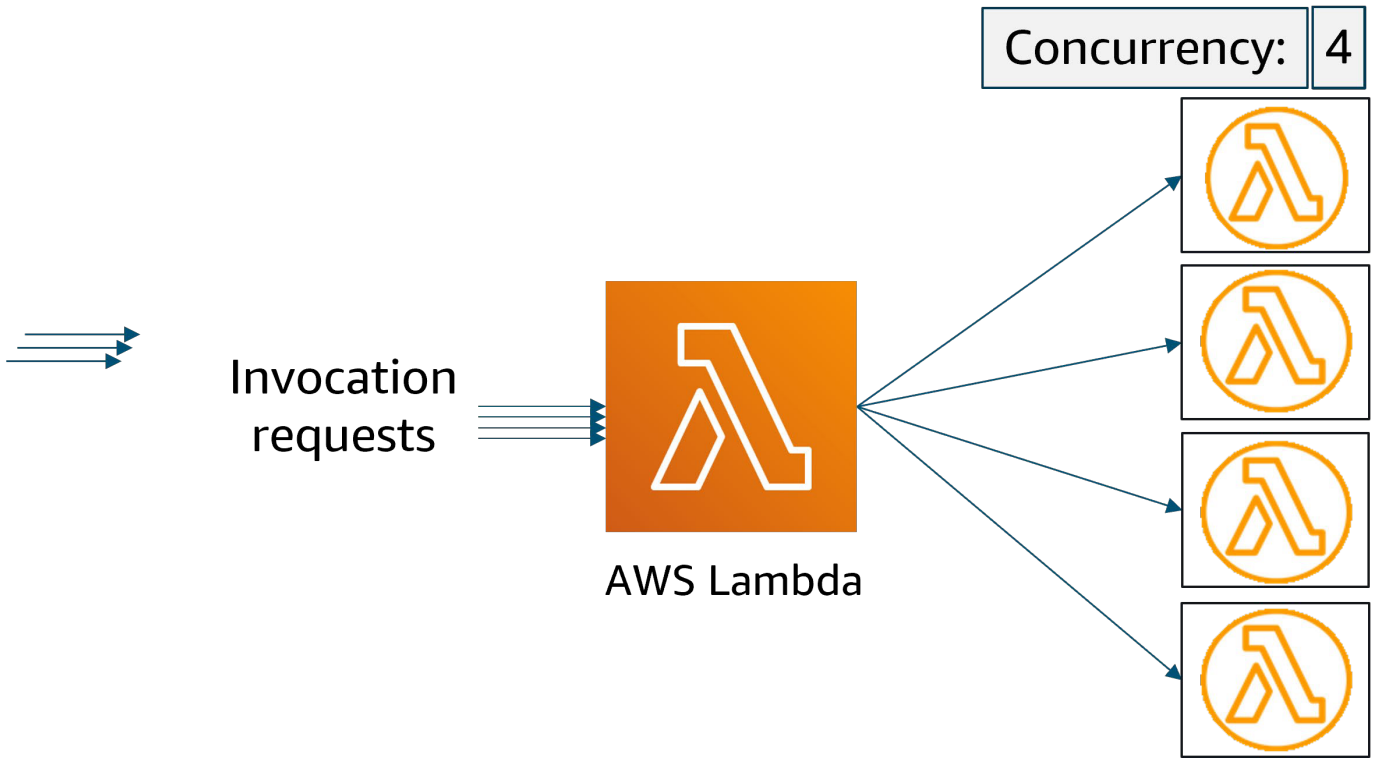
# Lambda functions




©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Lambda functions run in on-demand ephemeral environments

**Concurrency:** The number of function invocations that are running at one time




©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Section 3 key takeaways

- Lambda is a serverless compute service that integrates with other AWS services.
- You can use Lambda for many use cases including web applications, backends, and data processing.
- Lambda functions run in on-demand temporary environments.

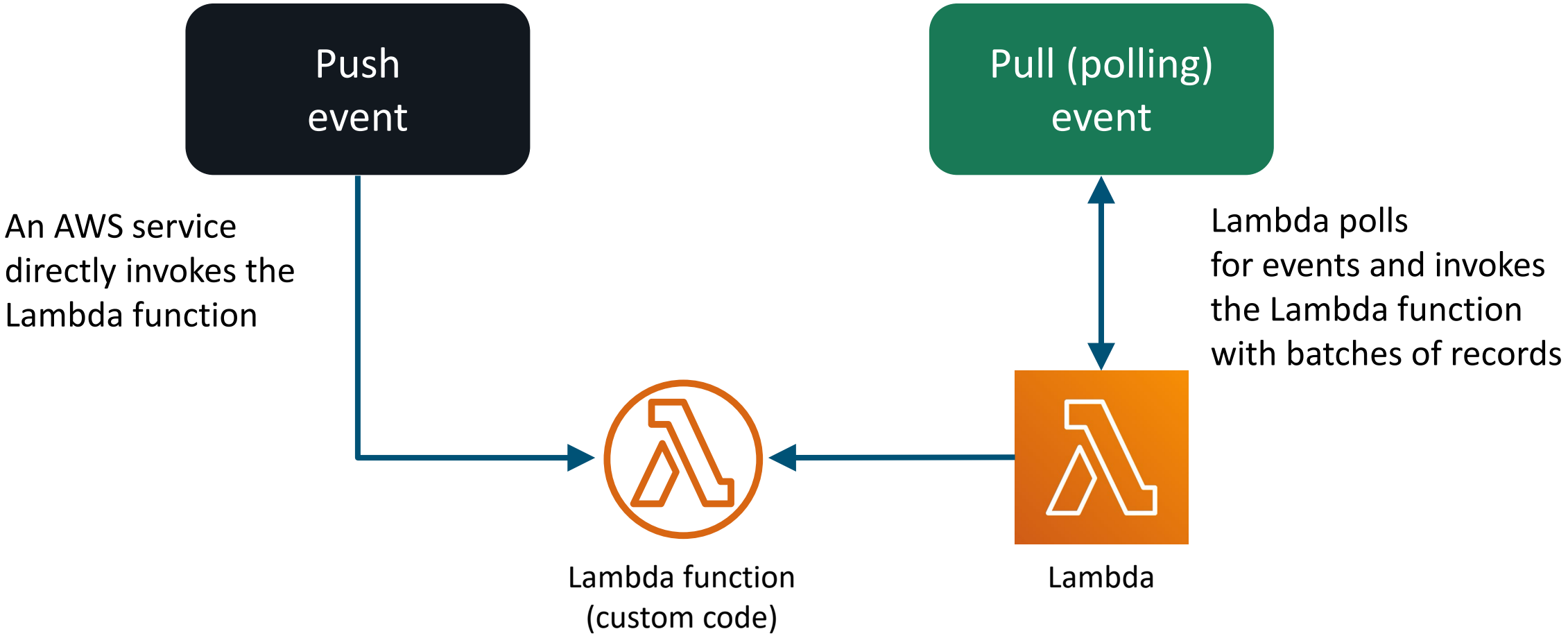
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Section 4: Invoking Lambda functions

## Module 7: Developing Event-Driven Serverless Solutions

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Push and pull models



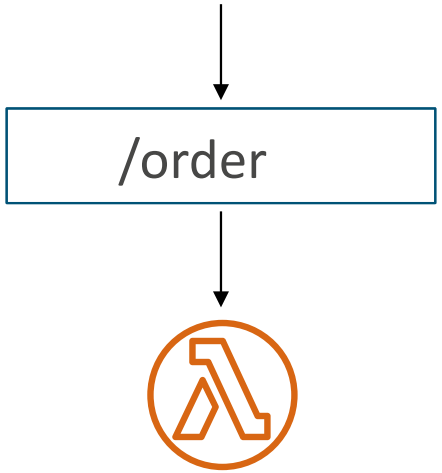

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Push event types

## Synchronous

The other service waits for a response from your function.

API Gateway




No retry built in

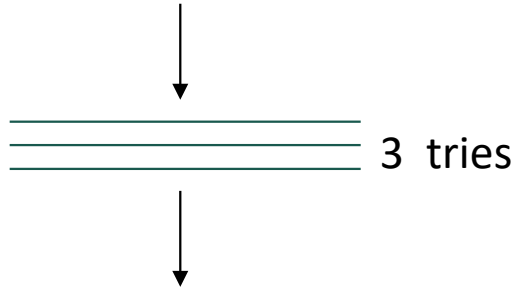
Invoke via API:
*RequestResponse*

Lambda function

## Asynchronous

Lambda queues the event before passing it to your function.

Amazon S3



0–2 retries

3 tries

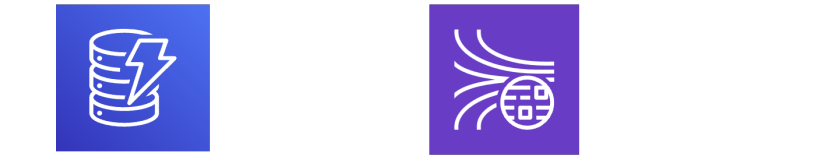
Invoke via API:
*Event*

Lambda function

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Pull (polling) event types

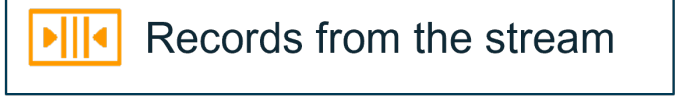
## Lambda polls a stream



Amazon DynamoDB Streams



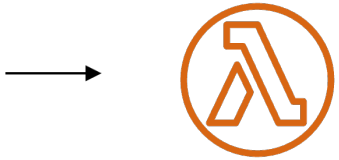
Amazon Kinesis Data Streams



shard

Errors in a shard block further processing

Lambda



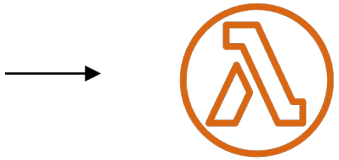
Lambda function

## Lambda polls a queue

Amazon Simple Queue Service (Amazon SQS)

Errors in a batch are returned to the queue

Lambda



Lambda function

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Invoking a function synchronously

Invoke

```
aws lambda invoke \
--function-name my-function \
--payload '{ "name": "Bob" }' \
response.json
```

Response

```
{"ExecutedVersion": "$LATEST","StatusCode": 200}
```

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Invoking a function asynchronously

Invoke

```
aws lambda invoke \
--function-name my-function \
--invocation-type Event \
--payload '{ "name": "Bob" }' \
response.json
```

Ack

```
{"StatusCode": 202}
```

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Section 4 key takeaways

- Lambda invokes Lambda functions by using one of these invocation models:
  - Synchronous
  - Asynchronous
  - Polling (event source mapping)
- Lambda has built-in handling that differs by event source type.

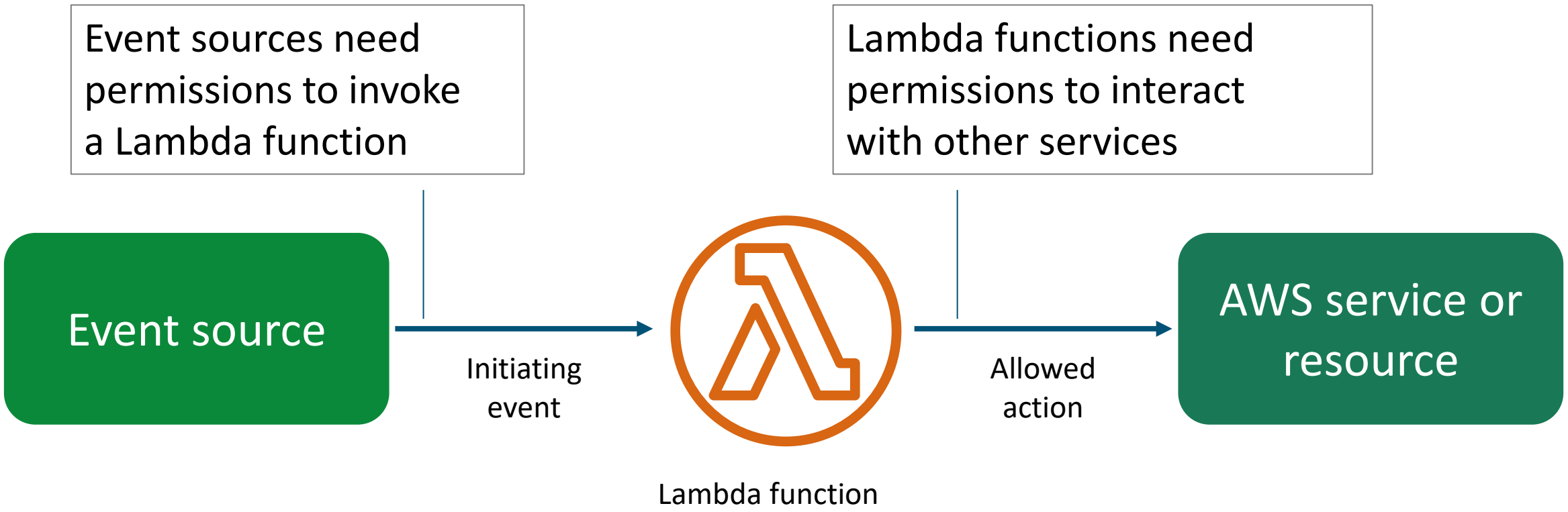
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Section 5: Setting permissions for Lambda

## Module 7: Developing Event-Driven Serverless Solutions

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

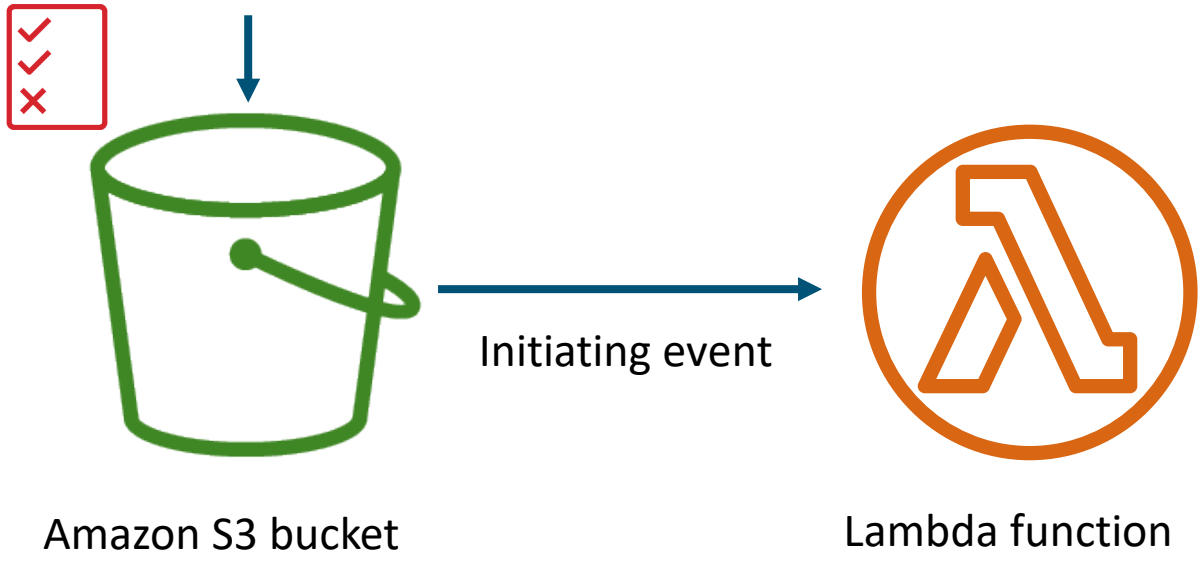
# Lambda permissions




©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Invocation permissions

An IAM resource policy gives permissions to invoke the function.




Resource policy:

- Associated with synchronous or asynchronous event source
- Allows the event source to take the *lambda:InvokeFunction* action

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

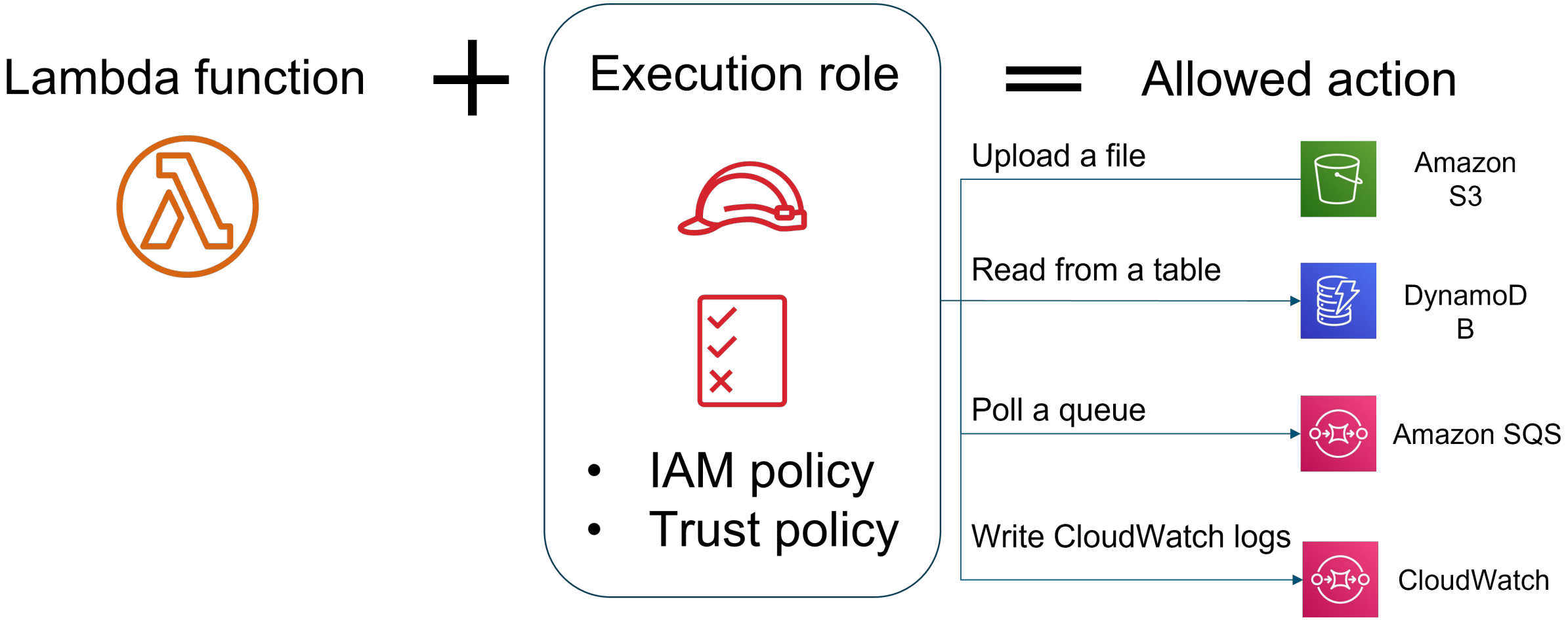
# Example resource (function) policy

```
{
        "Version": "2012-10-17",
        "Id": "default",
        "Statement": [
                {
                        "Sid": "lambda-fd269e28-988b-4d2b-96ae-eabcd7dc399c",
                        "Effect": "Allow",
                        "Principal": {
                                "Service": "s3.amazonaws.com"
                        },
                        "Action": "lambda:InvokeFunction",
                        "Resource": "arn:aws:lambda:function:myFirstFunction",
                        "Condition": {
                                "ArnLike": {
                                        "AWS:SourceARN": "arn:aws:s3:::myBucket1"
                                }
                        }
                }
        ]
}
```

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Lambda execution

The Lambda execution role specifies what the Lambda function is permitted to do.




©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Execution role example

IAM policy

```
{
        "Effect": "Allow",
        "Action": [
                "logs:CreateLogGroup",
                "logs:CreateLogStream",
                "logs:PutLogEvents"
        ],
        "Resource": "arn:aws:logs:*:*:*"
}
```

Trust policy

```
{
    "Effect": "Allow",
    "Principal": {
                "Service": "lambda.amazonaws.com"
      },
    "Action": "sts:AssumeRole"
}
```

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Adding permissions by using the AWS CLI

Give Amazon SNS permissions to invoke a function

```
aws lambda add-permission \
--function-name my-function \
--action lambda:InvokeFunction \
--statement-id sns \
--principal sns.amazonaws.com
```

Output

```
{
"Statement":
{
"Sid":"sns",
"Effect":"Allow",
"Principal":{
"Service":"sns.amazonaws.com"
},
"Action":"lambda:InvokeFunction",
"Resource":"arn:aws:lambda:us-east-2:123456789012:function:my-function"
}
}
```

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Section 5 key takeaways

- Lambda functions require permissions to be invoked and permissions to interact with other services.
- Both types of permissions are defined by using IAM or trust policies.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Section 6: Authoring and configuring Lambda functions

## Module 7: Developing Event-Driven Serverless Solutions

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# The function handler

Handler (event, context) method

## Event object

- Passes event information to the handler
- Uses a predefined object format for AWS integrations and events
- Can be tested with user-defined custom objects

## Context object

- Passes runtime information to the handler
- Includes, at a minimum, these methods or properties:
  - awsRequestId
  - getRemainingTimeInMillis()
  - logStreamName

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Lambda function handler example

event object

```
{
    "first_name": "John",
    "last_name": "Smith"
}
```

Handler

```
def lambda_handler(event, context):
    message = 'Hello {} {}!'.format(event['first_name'],
                                    event['last_name'])
    return {
        'message' : message
    }
```

response

```
{
    "message": "Hello John Smith!"
}
```

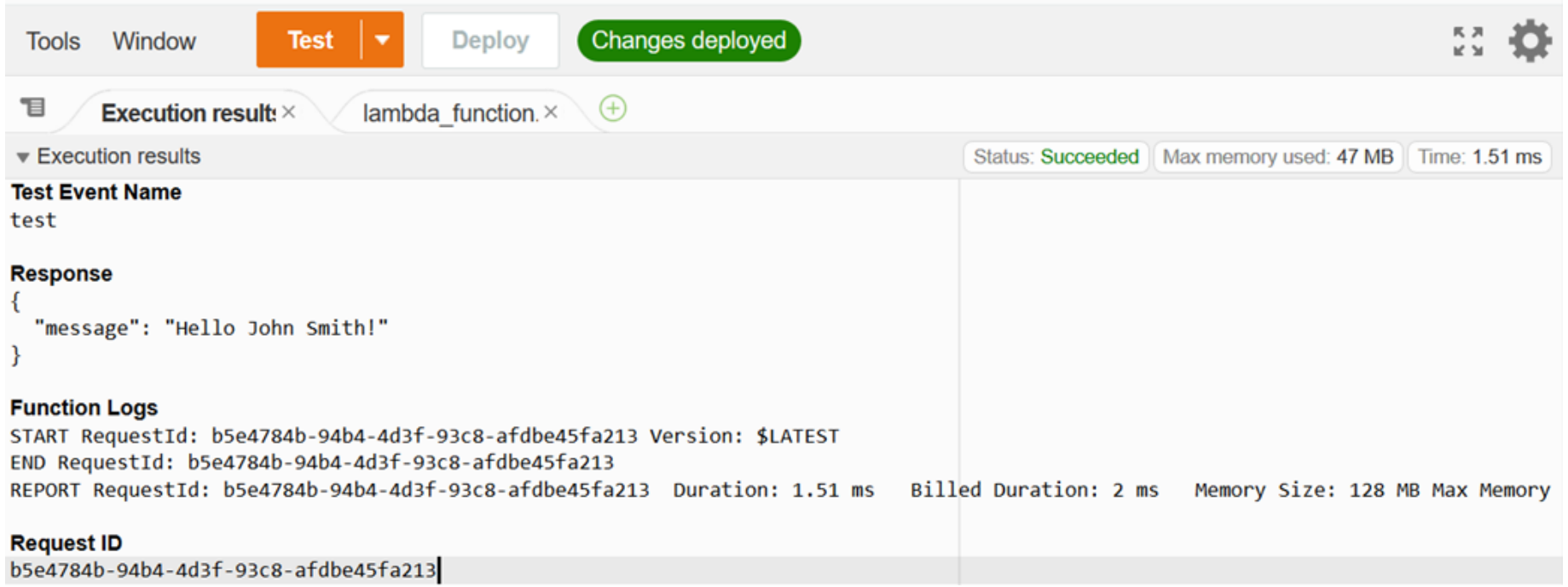
aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Example testing from the console

Create test event

Review results




©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Performance-related configurations

| Configuration | Description |
|---|---|
| Memory | Set the amount of memory and proportional CPU that is allocated to the function. Lambda allocates CPU power linearly in proportion to the memory that you set. Values from 128 MB–10,240 MB. |
| Timeout | Choose the maximum amount of time to let a function run before ending it if it has not completed. Values from 1 second up to 15 minutes. |
| Concurrency | Number of invocations of a function that can run at the same time. By default, you can have 1,000 concurrency invocations per Region for an account. This limit is a soft limit. Per function, you can set a limit to prevent overwhelming a downstream system or to reserve capacity from within the account pool for the function. |
| Provisioned concurrency | Number of Lambda environments to *keep warm*. When Lambda creates the temporary environment for your function, it will attempt to reuse it for additional invocations. This approach saves startup time. When Lambda must initialize new environments to run a function, an invocation might get a *cold start*, which creates startup latency. Use provisioned concurrency to avoid cold starts. Provisioned concurrency is priced separately. |
| Monitoring and operations | Settings to enable X-Ray (active tracing) and CloudWatch Lambda Insights (collect and aggregate Lambda function runtime performance metrics and logs). |

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Resource-related configurations

| Configuration | Description |
|---|---|
| Triggers | Event sources that invoke a function. |
| Permissions | The resources that have permissions to invoke the function and the permissions that the function has to interact with other resources. |
| Destinations | An SNS topic, SQS queue, other Lambda function, or EventBridge event bus. It receives invocation records from a function when it is successful (on success) or when it fails (on failure). |
| Asynchronous invocation | Settings for number of retry attempts on async invocations (0–2) and how long to keep an event waiting to be invoked (up to 6 hours). Also, configuration of a dead-letter queue for functions that continue to fail. |
| VPC | Settings to enable your Lambda function to access resources in a custom VPC. A custom VPC defines a private network of resources, such as databases, cache instances, or internal services. |
| State machines | Step Functions state machines that can invoke your function directly in at least one of the steps. |
| Database proxies | Settings that are used when connecting to an Amazon Relational Database Service (Amazon RDS) instance. These settings are used to set up a database proxy to manage connection pooling for database connections. |
| File systems | Settings to connect an Amazon Elastic File System (Amazon EFS) file system so that the function can access the file system at runtime. |

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Code-related configurations

| Configuration | Description |
| --- | --- |
| Runtime | Runtime that the function will use or language that the code will be written in. Choose from supported runtimes that are listed in the AWS Lambda Developer Guide or use a custom runtime. |
| Environment variables | Key-value pairs that are accessible from your function code. Environment variables are useful to store configuration settings without the need to change function code. |
| Tags | Labels that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs. |
| Code signing | Option to ensure that code has been signed by an approved source and has not been altered. |

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Design a Lambda function: Best practices for a Lambda environment

- Treat functions as stateless.
- Include only what you need.
- Reuse the temporary runtime environment.

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Design a Lambda function: Best practices for writing code

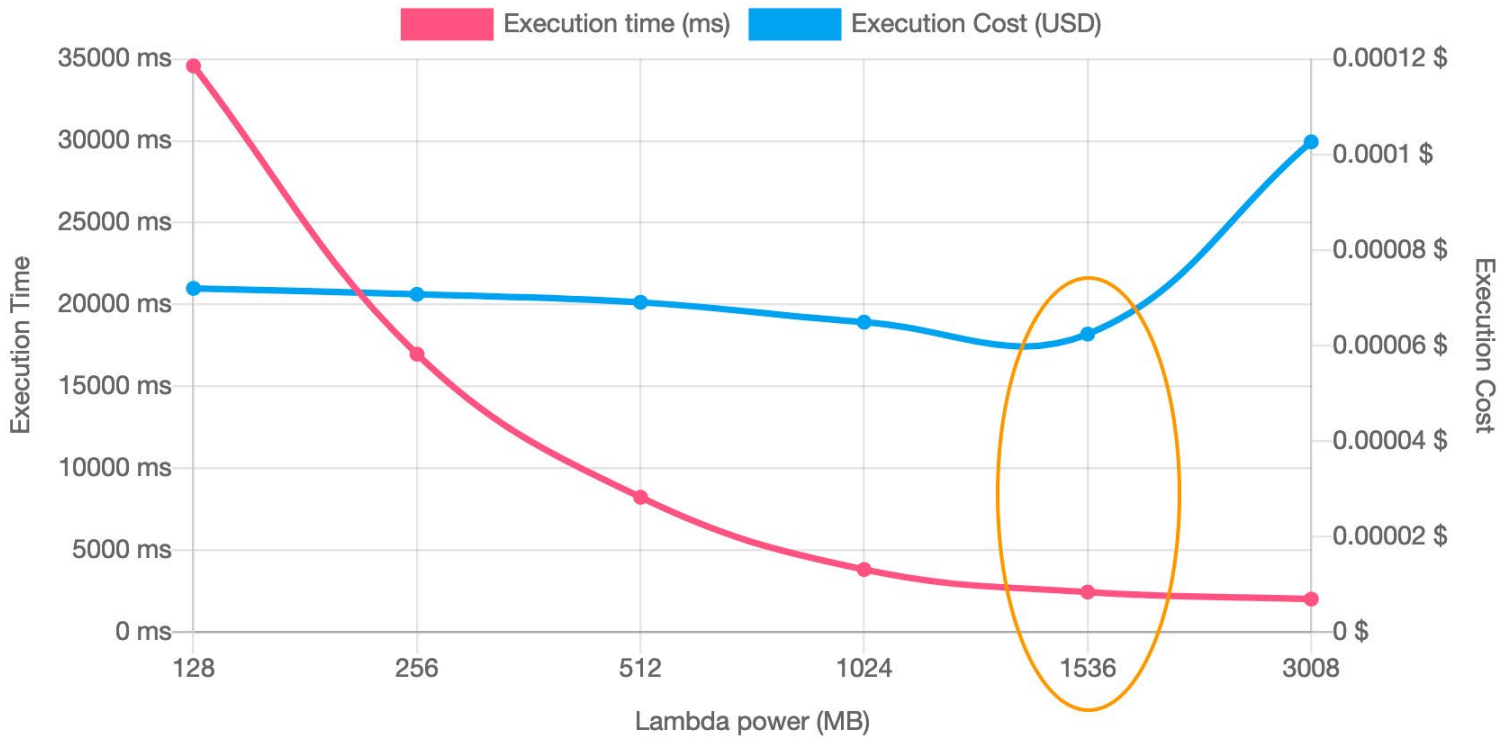
- Separate the core business logic (outside of the handler method).
- Write modular functions.
- Include logging statements.
- Include results information.
- Use environment variables.
- Avoid recursive code.
- Don't call one function from another.

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Tuning your function

- Higher memory configurations have a higher per-1-millisecond cost but might decrease duration costs and concurrency needs.
- Find the balance that optimizes for speed and cost.

Example results for two CPU-intensive functions, which become cheaper ***and*** faster with more power




Source: https://github.com/alexcasalboni/aws-lambda-power-tuning

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Section 6 key takeaways

- Lambda supports different languages and runtimes.
- The function handler is the entry point into the Lambda function.
- You can configure attributes including the amount of memory and function timeout.
- Follow best coding practices to create efficient Lambda functions.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Section 7: Deploying Lambda functions

## Module 7: Developing Event-Driven Serverless Solutions

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Deployment options

## .zip archive

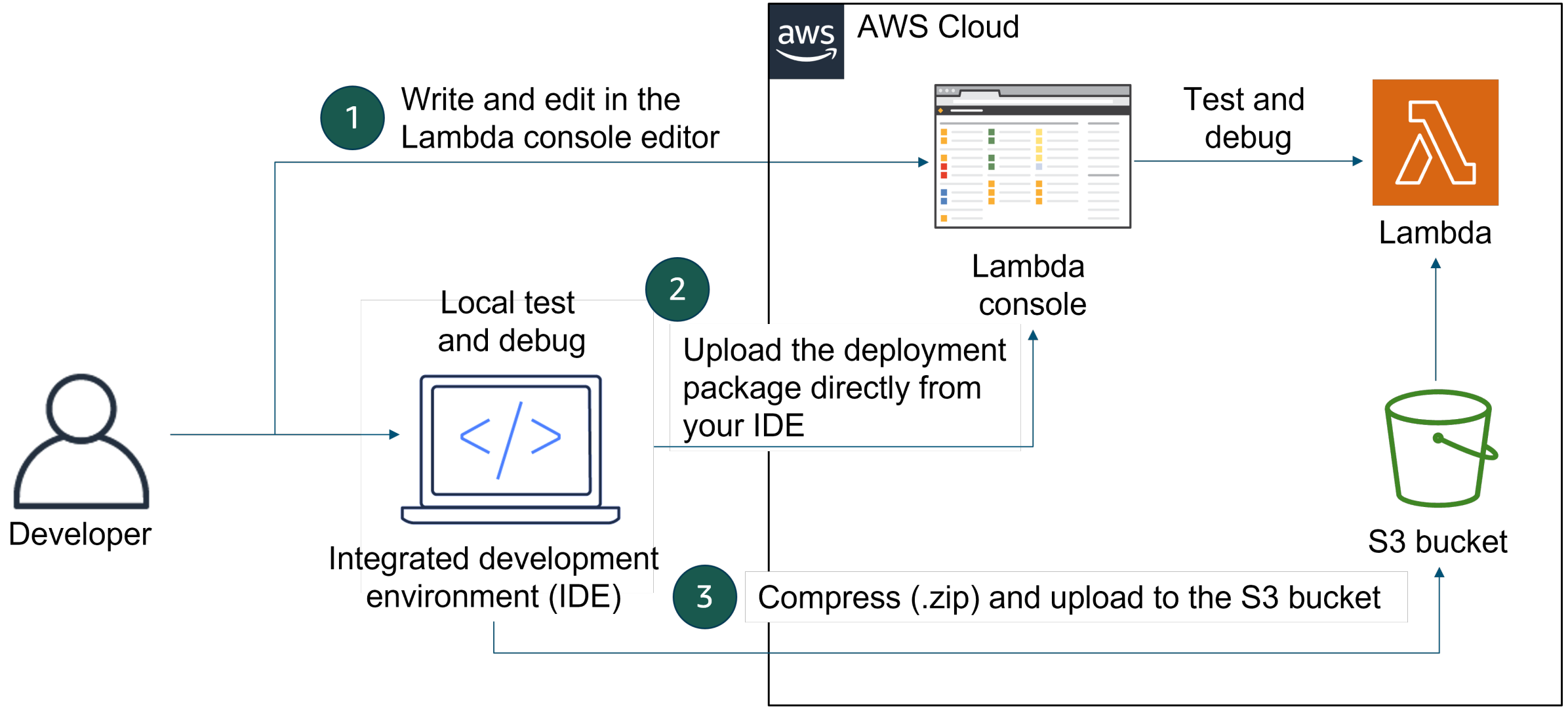
- Choose a **runtime** when you **create the function.**
- Compress files for application code and dependencies, and **upload them to Lambda.**

## Container image

- Choose a **runtime and linux distribution** when **creating the image.**
- Package your code and dependencies as a **container image**.
- **Upload** the image to your **container registry** that is hosted on Amazon Elastic Container Registry (Amazon ECR).

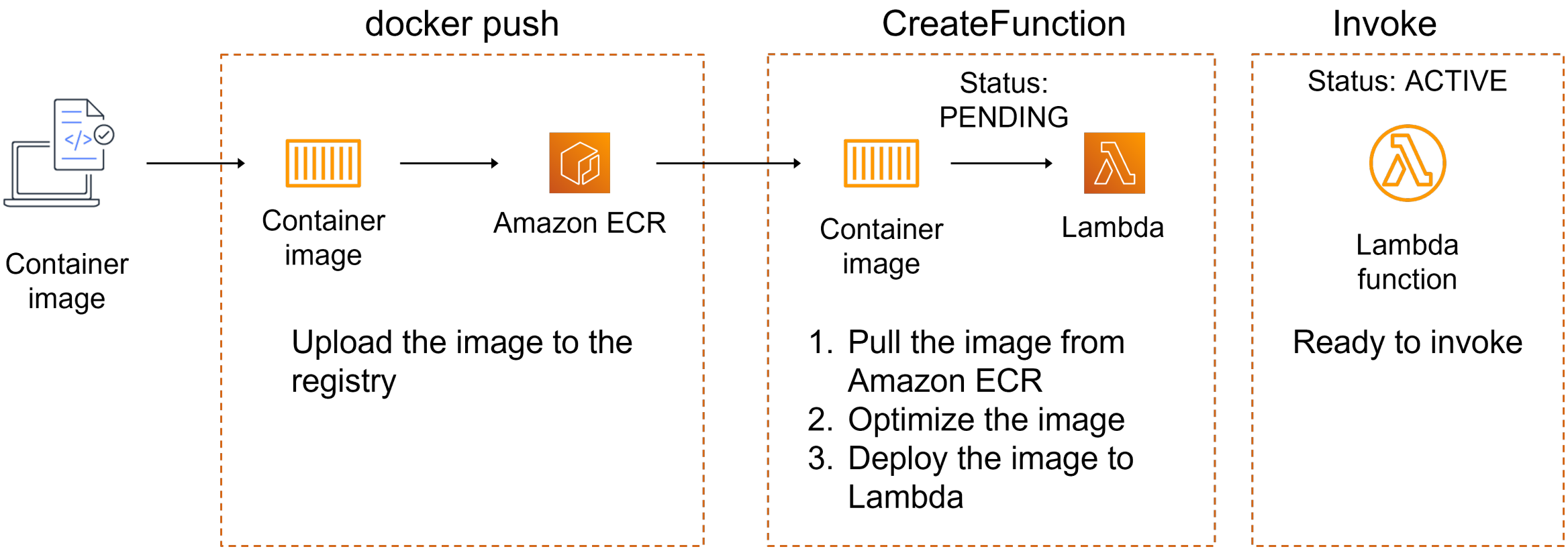
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Uploading .zip deployment packages




©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Deploying as a container image




©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

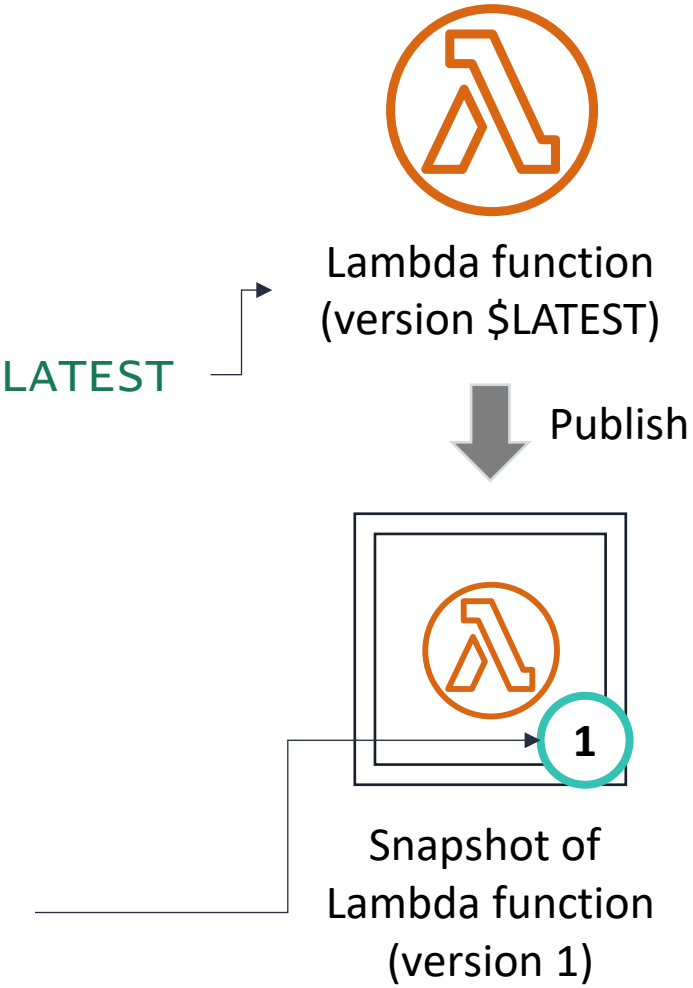
# Versioning

- Immutable copies of Lambda function code and configuration
- Create: $LATEST version

```
arn:aws:lambda:aws-region:acct-id:function:helloworld:$LATEST
```

- Publish: Snapshot copy of $LATEST
- Each version has its own Amazon Resource Name (ARN) with a new sequential version number

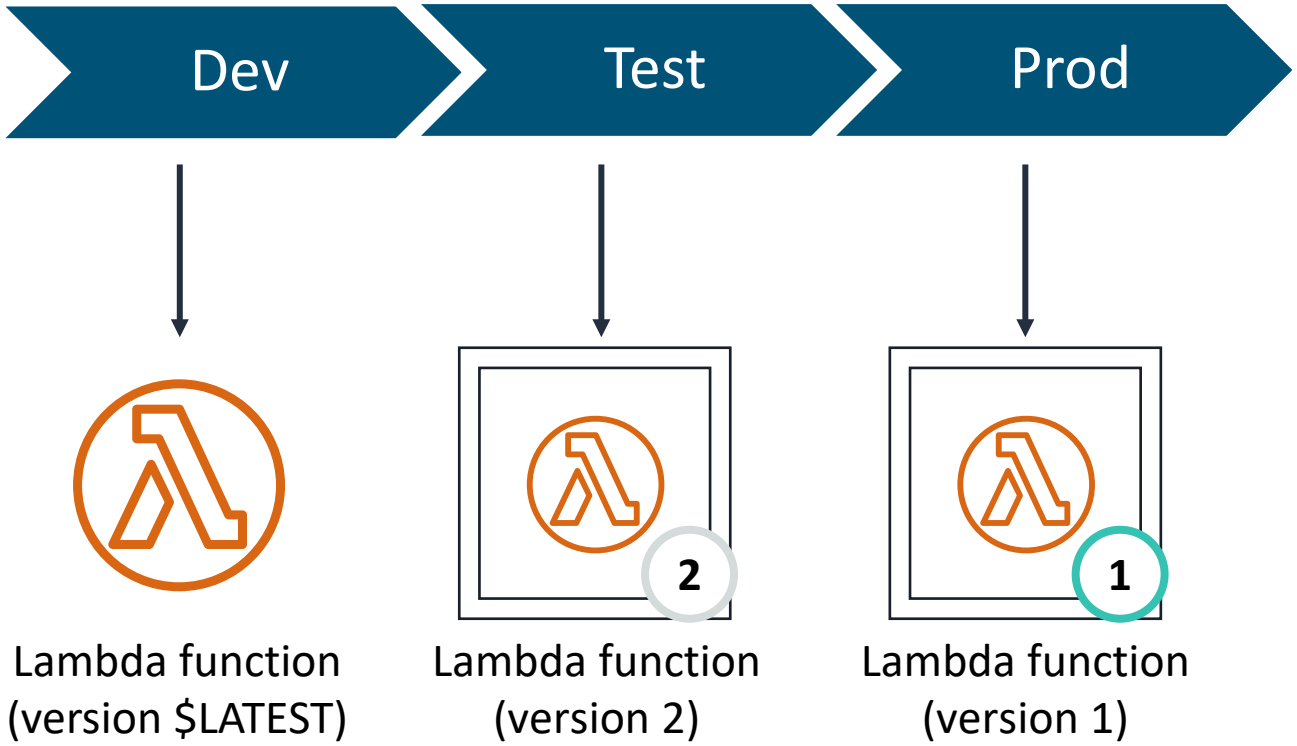
```
arn:aws:lambda:aws-region:acct-id:function:helloworld:1
```




©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Aliases

- Mutable pointers to versions
- Each alias has its own ARN




Lambda function test alias:
`arn:aws:lambda:aws-region:acct-id:function:helloworld:test`

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Example of using versioning and aliases

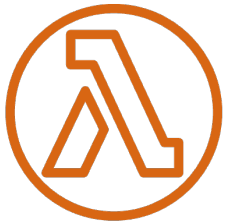
## Without aliases:

Amazon S3

arn:aws:lambda:aws-region:acct-id:function:helloworld:$LATEST

arn:aws:lambda:aws-region:acct-id:function:helloworld:*1*

Event source mapping that uses the Lambda function ARN (update with each version)



Lambda function (version $LATEST)

Publish

## With aliases:

Amazon S3

Event source mapping that uses the alias ARN (don't need to update with each version)

arn:aws:lambda:aws-region:acct-id:function:helloworld:PROD

Snapshot of Lambda function (version 1)

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Custom runtimes

- With custom runtimes you can use **other languages** in Lambda.
- Lambda provides a **runtime interface** for getting events and sending responses.
- The **bootstrap file** is the runtime's entry point.
- You can deploy a custom runtime next to your function code or **in a layer**.

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Lambda layers

Centrally manage code and data that are shared across multiple functions

- Reduce the size of deployment packages
- Speed up deployments
- Limits:
  - Up to five layers
  - 250 MB

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Example deploying from the AWS CLI

Create function

```
aws lambda create-function \
--function-name my-function \
--runtime nodejs10.x \
--zip-file fileb://my-function.zip \
--handler my-function.handler \
--role arn:aws:iam::123456789012:role/service-role/MyTestFunction-role-tges6bf4
```

Output

```
{
"TracingConfig": {
"Mode": "PassThrough"
},
"CodeSha256": "PFn4S+er27qk+UuZSTKEQfNKG/XNn7QJs90mJgq6oH8=",
"FunctionName": "my-function",
"CodeSize": 308,
"RevisionId": "873282ed-4cd3-4dc8-a069-d0c647e470c6",
"MemorySize": 128,
"FunctionArn": "arn:aws:lambda:us-west-2:123456789012:function:my-function",
"Version": "$LATEST",
"Role": "arn:aws:iam::123456789012:role/service-role/MyTestFunction-role-zgur6bf4",
"Timeout": 3,
"LastModified": "2019-08-14T22:26:11.234+0000",
"Handler": "my-function.handler",
"Runtime": "nodejs10.x",
}
```

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Example of updating a function from the AWS CLI

Update function

```
aws lambda update-function-code \
--function-name my-function \
--zip-file fileb://my-function.zip
```

Output

```
{
"FunctionName": "my-function",
"LastModified": "2019-09-26T20:28:40.438+0000",
"RevisionId": "e52502d4-9320-4688-9cd6-152a6ab7490d",
"MemorySize": 256,
"Version": "$LATEST",
"Role": "arn:aws:iam::123456789012:role/service-role/my-function-role-uy3l9qyq",
"Timeout": 3,
"Runtime": "nodejs10.x",
"TracingConfig": {
"Mode": "PassThrough"
},
"CodeSha256": "5tT2qgzYUHaqwR716pZ2dpkn/0J1FrzJmlKidWoaCgk=",
"CodeSize": 304,
"FunctionArn": "arn:aws:lambda:us-west-2:123456789012:function:my-function",
"Handler": "index.handler"
}
```

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Section 7 key takeaways

- Lambda has built-in versioning so that you can create immutable versions of your code.
- You can use aliases to point at different versions of your Lambda function.
- Lambda provides popular runtimes, but you can create custom runtimes.
- Lambda layers are libraries of code that you create and which other Lambda functions can call.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Section 8: Monitoring and debugging tools for application developers

## Module 7: Developing Event-Driven Serverless Solutions

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Modern applications challenge traditional monitoring approaches

Short-lived resources

More devices, services, and data

Faster release velocity

User experience

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Amazon CloudWatch

Amazon CloudWatch

CloudWatch collects monitoring and operational data in the form of logs, metrics, and events.

- Get a **unified view** of AWS resources, applications, and services
- Review logs as a **flow of events ordered by time**
- **Query** log data interactively with **CloudWatch Logs Insights**
- Use **default operational metrics** from AWS services
- **Alarm** based on metric thresholds and initiate **automated actions**

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Example of Lambda logging

Log group

Log stream

CloudWatch > Log Groups > /aws/lambda/FAQ > 2020/08/27/[$LATEST]05419d29f9014fc3b1634b1e9968ef28

| Time | Message |
|---|---|
| 19:35:32 | START RequestId: 41c8dcf0-ff10-4d96-af2a-1ab5e346c937 Version: $LATEST |
| 19:35:32 | 2020-08-27T19:35:32.553Z 41c8dcf0-ff10-4d96-af2a-1ab5e346c937 INFO … |
| 19:35:32 | 2020-08-27T19:35:32.575Z 41c8dcf0-ff10-4d96-af2a-1ab5e346c937 INFO … |
| 19:35:32 | END RequestId: 41c8dcf0-ff10-4d96-af2a-1ab5e346c937 |
| 19:35:32 | REPORT RequestId: 41c8dcf0-ff10-4d96-af2a-1ab5e346c937 Duration: 64.40 ms Billed Duration: 100 ms Memory Size: 128 MB Max Memory Used: 68 MB Init Duration: 162.85 ms |
| 19:35:36 | START RequestId: a23cbea6-3053-4b51-8721-ff11aba24e33 Version: $LATEST |
| … | |

Log events

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# AWS X-Ray

AWS X-Ray

X-Ray traces requests that travel through your application and shows a map of your application's underlying component.

- **One-click** enablement for **Lambda functions** and integration with other AWS services
- **X-Ray SDK** to capture metadata for **API calls** that are made to AWS services with the **AWS SDK**
- Visual **detection** of **latency distribution** and quick **isolation** of **outliers** and trends
- Easier **debugging** of application code
- **Filtering** and **grouping** of requests by error type

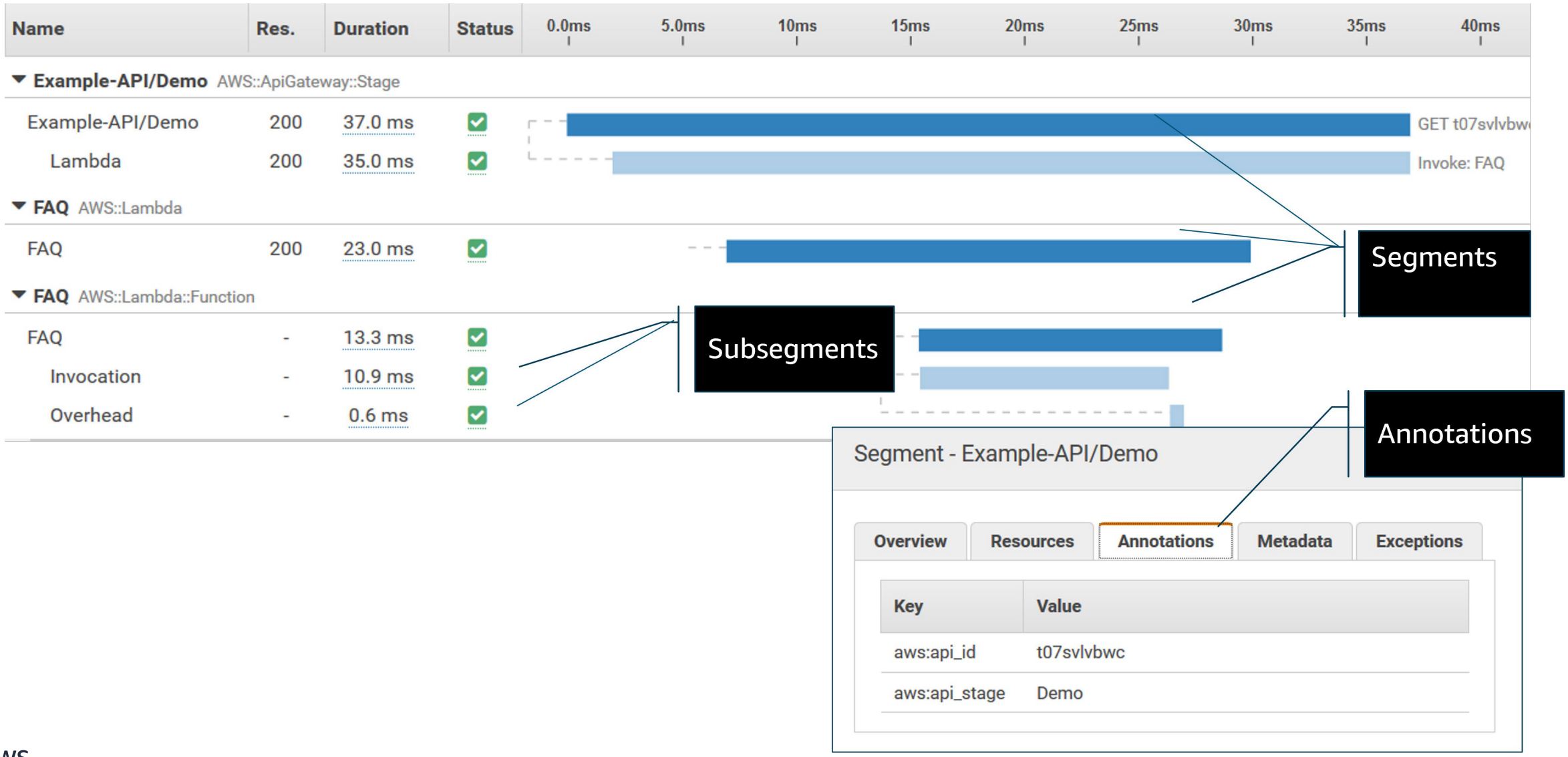
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Parts of an X-Ray trace

| Element | Description |
| --- | --- |
| Segments | Data about the work done by compute resources that run your application, including:<br>• Resource name<br>• Details about the request and the work that is done<br>• Subsegments |
| Subsegments | Breakdown of segment data into greater detail, including:<br>• More granular timing<br>• Details about downstream calls<br>• Inferred segments for downstream services<br>• Annotations and metadata |
| Annotations | Key-value pairs that can be indexed and used with filter expressions to group traces in the console for easier analysis |
| Metadata | Key-value pairs of any type that are not indexed but can be used to store data in the trace that you won't use for searching traces |

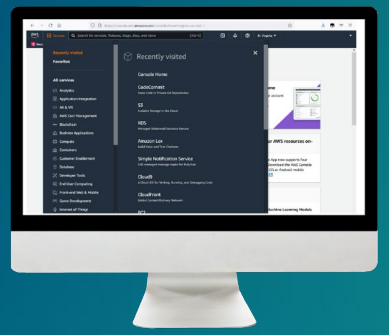
aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Example X-Ray trace




©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Demonstration: Using X-Ray with Lambda



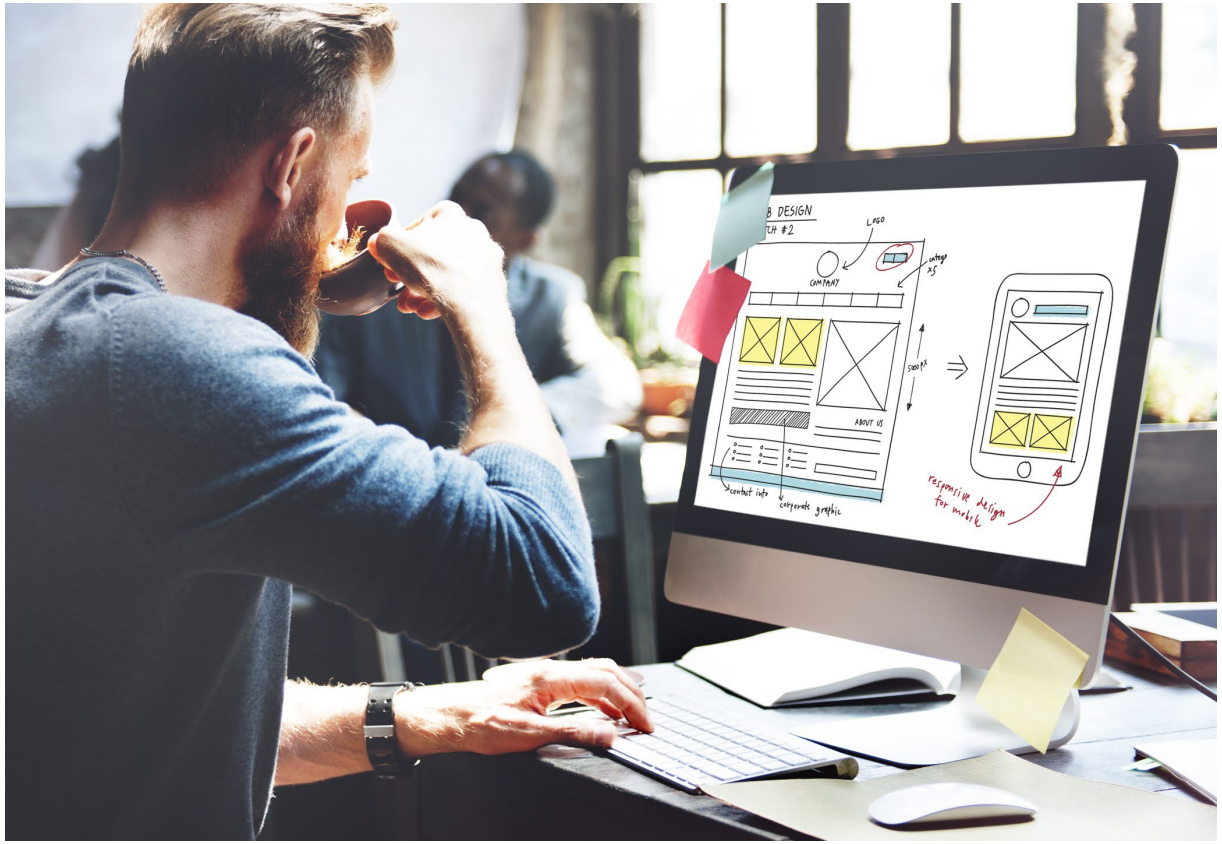
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Section 8 key takeaways

- Modern applications require more than traditional operations monitoring.
- CloudWatch provides built-in logging and metrics to monitor your Lambda functions.
- With X-Ray, you can trace requests across your application to find trouble spots quickly.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Lab 7.1: Creating Lambda Functions Using the AWS SDK for Python



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Lab: Scenario

In this lab, you again play the role of Sofía. You replace the mock endpoints that you created in the previous lab with real endpoints so that the web application can connect to the database. You will use Lambda to make this connection between the REST API and the data that is stored in DynamoDB.

aws

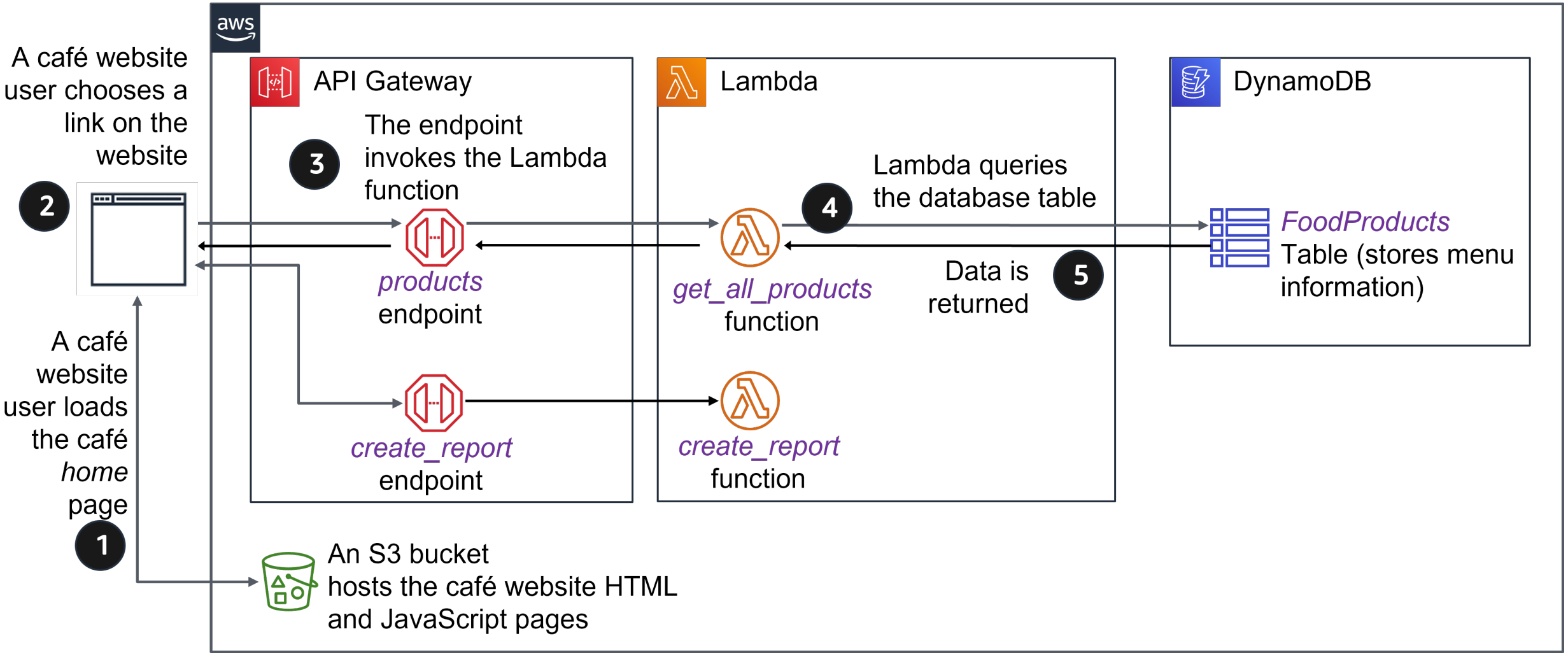
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Lab: Tasks

1. Preparing the development environment
2. Creating the get_all_products Lambda function
3. Configuring the REST API GET method to invoke the Lambda function
4. Creating the create_report Lambda function
5. Configuring the REST API POST method to invoke the Lambda function
6. Testing the integration by using the café website

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Lab: Final product




©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

~ 90 minutes

# Begin Lab 7.1: Creating Lambda Functions Using the AWS SDK for Python

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Lab debrief: Key takeaways

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Module wrap-up

## Module 7: Developing Event-Driven Serverless Solutions

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Module summary

In summary, in this module, you learned how to:

- Explain serverless computing
- Describe how Lambda works
- Recognize Lambda invocation models
- Identify how to use IAM to grant Lambda permissions
- Indicate the steps to author and configure a Lambda function
- Explain how to deploy Lambda functions
- Identify why X-Ray is a critical developer tool
- Create Lambda functions

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Complete the knowledge check

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Sample exam question

A developer is testing an application locally and has deployed it to Lambda. To remain under the package size limit, they did not include the dependencies in the deployment file. When testing the application remotely, the function does not run because of missing dependencies.

Which approach would best resolve the issue?

Identify the key words and phrases before continuing.

**The following are the key words and phrases:**

- **Under the package size limit**
- **Dependencies**

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Sample exam question: Response choices

A developer is testing an application locally and has deployed it to Lambda. To remain **under the package size limit**, they did not include the **dependencies** in the deployment file. When testing the application remotely, the function does not run because of missing dependencies.

Which approach would best resolve the issue?

| Choice | Response |
|---|---|
| **A** | Use the Lambda console editor to update the code and include the missing dependencies. |
| **B** | Create an additional .zip file with the missing dependencies, and include the file in the original Lambda deployment package. |
| **C** | Add references to the missing dependencies in the Lambda function's environment variables. |
| **D** | Create a layer that contains the missing dependencies, and attach it to the Lambda function. |

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Sample exam question: Answer

The correct answer is D.

| Choice | Response |
| --- | --- |
| **A** | Use the Lambda console editor to update the code and include the missing dependencies. |
| **B** | Create an additional .zip file with the missing dependencies, and include the file in the original Lambda deployment package. |
| **C** | Add references to the missing dependencies in the Lambda function’s environment variables. |
| **D** | **Create a layer that contains the missing dependencies, and attach it to the Lambda function.** |

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Additional resources

- AWS Lambda Developer Guide:
  - Best Practices for Working with AWS Lambda Functions
  - Using AWS Lambda with Other Services
  - AWS Lambda Execution Environment
  - Troubleshooting Issues in Lambda
  - Monitoring Functions in the AWS Lambda Console
  - Accessing Amazon CloudWatch logs for AWS Lambda
  - Using AWS Lambda with AWS X-Ray
- Amazon CloudWatch Logs User Guide:
  - Analyzing Log Data with CloudWatch Logs Insights

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Thank you

Corrections, feedback, or other questions?
Contact us at https://support.aws.amazon.com/#/contacts/aws-academy.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.